

2. (a) retry tidak boleh dilakukan ketika request bersifat non-idempotent, seperti pembayaran atau transfer, karena bisa menyebabkan proses yang sama dijalankan dua kali, misalnya double charge. Selain itu, retry juga tidak perlu dilakukan jika error berasal dari sisi client seperti HTTP 400, 401, atau 403, karena masalahnya ada pada request, bukan pada service. Dalam kondisi service benar-benar down atau overload, retry berlebihan juga sebaiknya dihindari karena justru dapat memperparah beban sistem.

(b). Retry digunakan untuk mencoba kembali request yang gagal karena gangguan sementara, misalnya timeout atau koneksi jaringan yang tidak stabil. Jadi sistem masih mencoba berharap request berikutnya berhasil.

Sedangkan circuit breaker digunakan untuk menghentikan sementara request ke service yang sedang bermasalah. Jika terlalu banyak request gagal, circuit breaker akan membuka (open state) dan request berikutnya langsung ditolak sementara agar service tidak semakin overload.)

(c). Retry tanpa jitter atau backoff memiliki risiko besar karena semua client bisa melakukan retry secara bersamaan. Hal ini dapat menyebabkan lonjakan traffic mendadak atau thundering herd problem, sehingga service yang awalnya hanya lambat menjadi benar-benar down.

Selain itu, retry terus-menerus tanpa jeda bisa menyebabkan:

- peningkatan beban CPU dan connection pool,
- retry storm,
- cascading failure ke service lain,
- dan memperlambat proses recovery service.

Karena itu biasanya retry dikombinasikan dengan exponential backoff dan jitter agar request retry dilakukan secara bertahap dan tidak bersamaan.

3. (a) Informasi minimum yang wajib ada di setiap log menurut saya adalah timestamp, traceId atau correlationId, nama service, endpoint yang dipanggil, status request, dan message dari proses tersebut. Jika memungkinkan juga ditambahkan userId atau requestId agar lebih mudah melakukan investigasi terhadap kasus tertentu.

Karena di arsitektur microservice, satu request bisa melewati banyak service, jadi tanpa informasi tersebut akan sulit melakukan tracking ketika terjadi masalah seperti data hilang.

(b). Cara menelusuri satu request dari awal sampai akhir biasanya menggunakan traceId yang sama di seluruh service.

Jadi ketika request pertama masuk, misalnya dari API Gateway, system akan generate traceId lalu traceId tersebut diteruskan ke semua microservice melalui header request. Setelah itu setiap service wajib mencatat traceId yang sama di log mereka.

Dengan cara itu kita bisa mencari seluruh alur request hanya menggunakan satu traceId, mulai dari request masuk sampai response terakhir.

(c). Jika traceId tidak konsisten antar service, dampaknya proses tracing menjadi sulit karena log antar service tidak bisa dihubungkan.

Akibatnya:

- sulit mengetahui request berjalan ke mana saja,
- root cause analysis menjadi lama,
- debugging harus dilakukan manual satu per satu,
- dan monitoring distributed system menjadi tidak efektif.

Dalam kasus “data hilang”, kita juga akan kesulitan menentukan di service mana data tersebut gagal diproses karena alur request tidak terlihat secara utuh.

4. (a). Untuk memastikan consumer bersifat idempotent, consumer harus mampu memproses event yang sama lebih dari satu kali tanpa menyebabkan duplicate data atau perubahan yang salah.

Biasanya caranya dengan menyimpan eventId atau messageId dari event yang sudah diproses. Jadi sebelum consumer memproses event, system akan mengecek apakah event tersebut sudah pernah diproses atau belum. Kalau sudah, event akan di-skip.

Contohnya pada event CustomerUpdated, walaupun event terkirim dua kali karena retry dari broker, data customer tetap hanya ter-update satu kali.

(b). Ordering sebagian dijamin oleh broker, tetapi tetap perlu dijaga di sisi consumer dan desain sistem.

(c). Jika ordering tidak terjaga, data bisa menjadi tidak konsisten karena event lama bisa menimpa data yang lebih baru.

5. (a). Cache harus di-invalidate ketika data asli di database berubah atau sudah tidak valid lagi. Tujuannya agar data di cache tetap konsisten dengan data sebenarnya.

Biasanya cache di-invalidate pada kondisi:

- setelah proses create, update, atau delete data,
- ketika TTL (time to live) habis,
- atau saat terjadi perubahan penting yang harus langsung tercermin ke user.

(b). Menurut saya cache sebaiknya tidak dijadikan source of truth. Source of truth tetap database utama karena cache hanya berfungsi untuk mempercepat akses data.

Cache bersifat sementara dan bisa:

- expired,
- hilang,
- atau tidak sinkron dengan database.

Kalau cache dijadikan source of truth, ada risiko data inconsistency dan kehilangan data ketika cache restart atau failure.

(c). Cache stampede terjadi ketika cache expired lalu banyak request langsung mengakses database secara bersamaan. Akibatnya database bisa overload.

Cara mencegahnya biasanya:

- menggunakan TTL yang berbeda-beda (randomized TTL),
- menggunakan locking sehingga hanya satu request yang refresh cache,
- menerapkan stale cache,
- atau menggunakan background refresh.

Contohnya dengan locking:

- request pertama melakukan refresh ke database,
- request lain menunggu hasil cache baru,
- sehingga database tidak terkena lonjakan query yang sama secara bersamaan.

Cara ini penting terutama pada service dengan traffic tinggi agar database tetap stabil dan tidak menjadi bottleneck.

6. (a) Menurut saya penggunaan API key saja belum cukup untuk komunikasi antar microservice, terutama di production system dengan traffic dan service yang banyak.

Karena API key biasanya bersifat static dan jika bocor, service lain bisa mengakses API tanpa kontrol yang kuat. Selain itu API key juga tidak memiliki identitas detail seperti siapa service yang melakukan request dan biasanya tidak memiliki mekanisme expiry.

API key masih bisa digunakan sebagai tambahan security, tetapi sebaiknya dikombinasikan dengan mekanisme lain seperti:

- HTTPS/TLS,
- JWT,
- mutual TLS (mTLS),
- atau service authentication antar service.

(b) Untuk mencegah service palsu mengakses API internal, biasanya dilakukan authentication dan verification antar service.

Beberapa cara yang umum digunakan:

- menggunakan mutual TLS (mTLS) sehingga kedua service saling memverifikasi certificate,
- menggunakan signed JWT khusus antar service,
- membatasi akses hanya dari internal network atau service mesh,
- dan menerapkan whitelist service identity.

Dengan cara itu, hanya service yang terdaftar dan trusted yang bisa mengakses API internal.

(c) Risiko penggunaan JWT tanpa expiry pendek adalah token yang bocor bisa digunakan dalam waktu lama oleh pihak yang tidak berhak.

Dampaknya cukup berbahaya karena attacker bisa:

- terus mengakses internal API,
- melakukan impersonasi service,
- mengambil data,
- atau menjalankan request ke service lain tanpa terdeteksi.

Selain itu, token yang terlalu lama expired juga menyulitkan proses revocation ketika terjadi compromise.

Karena itu biasanya JWT untuk komunikasi internal dibuat:

- memiliki expiry pendek,
- menggunakan refresh mechanism,
- dan dikombinasikan dengan TLS atau mTLS agar lebih aman.